

× PESSIMISTIC

SECURITY ANALYSIS

by Pessimistic

This report is public

June 12, 2026

Abstract 2
Disclaimer 2
Summary 2
General recommendations 3
Project overview 4
Project description 4
Codebase update #1 4
Audit process 5
Manual analysis 6
Critical issues 6
C01. Reentrancy risks during minting and interaction with malicious tokens (fixed) 6
Medium severity issues 8
M01. New liquidity providers can capture historical borrower interest (commented) 8
M02. New liquidity providers can claim historical Uniswap fees (fixed) 9
M03. Provider's position is not synchronized with latest Uniswap fees (fixed) 9
M04. Typo in mint refund condition (fixed) 10
M05. Insufficient documentation (commented) 11
Low severity issues 12
L01. Division by zero (fixed) 12
L02. Double check (fixed) 12
L03. Double rounding (fixed) 12
L04. Immutable variable (fixed) 13
L05. Inconsistent and misleading naming reduces code readability (partially fixed) 13
L06. Inconsistent token ordering in parameter passing (commented) 14
L07. Missing event (fixed) 14
L08. Missing functionality (commented) 14
L09. Moving logic to constructor (fixed) 15
L10. Multiple reads from storage (partially fixed) 15
L11. Overcollateralization increases borrower losses (commented) 15
L12. Redundant check (fixed) 16
L13. Griefing via rounding in repeated interest updates (fixed) 16
Notes 16
N01. Unused import (fixed) 16

ABSTRACT

This report evaluates the security of the smart contracts in the Kommodo project. The objective was to identify and document security vulnerabilities within the platform's smart contracts.

DISCLAIMER

This audit makes no warranties or guarantees as to the security of the code. A single audit is insufficient to ensure complete security. We strongly recommend conducting multiple independent audits and implementing a public bug bounty program to enhance smart contract security. Additionally, this security audit should not be construed as investment advice.

SUMMARY

This report assesses the security of the Kommodo smart contracts. The audit process is detailed in the following section.

The initial audit identified one critical issue:
Reentrancy risks during minting and interaction with malicious tokens. The audit also revealed several medium severity issues:
New liquidity providers can capture historical borrower interest,
New liquidity providers can claim historical Uniswap fees,
Provider's position is not synchronized with latest Uniswap fees,
Typo in mint refund condition, Insufficient documentation. Moreover, several low severity issues were found.

After the initial audit, the codebase was updated. Most issues were fixed (C01, M02, M03 and M04) or commented on (M05). M01 was partially resolved and commented on. However, M01 and M05 remain relevant. All low severity issues were either fixed or acknowledged. Additionally, one low severity issue and one informational note were identified during the recheck and immediately fixed by the developers.

Overall, the code quality is good and the system can operate correctly if used as intended. However, the underlying scenario described in M01, as well as the documentation gaps identified in M05, remain present and should be addressed to improve clarity and reduce potential misuse.

GENERAL RECOMMENDATIONS

We recommend addressing the remaining unresolved issues and improving the documentation. While the current code coverage is high, we suggest further improving branch coverage to increase confidence in edge-case behavior.

PROJECT OVERVIEW

Project description

For this audit, we reviewed the Kommodo project from a public GitHub repository, commit 6f67da9397c5503806a055b118ca016e750579c7.

The scope of the audit included **Connector.sol**, **Kommodo.sol**, **KommodoFactory.sol**, **NonfungibleLendManager.sol**.

The provided documentation included the following link.

The contracts will be deployed to Ethereum Mainnet only.

All 55 tests pass successfully. The code coverage could not be measured.

The total lines of code (LOC) of audited sources is 670.

Codebase update #1

After the initial audit, the codebase was updated. For the recheck, we were provided with commit 4b37bbadb9d64fdf7b6de9c8a2210c8382267ac9.

58 out of 58 tests passed. The code coverage was 99.85%.

The critical issue was fixed. Three medium severity issues were fixed. Low severity issues were either fixed or acknowledged. Additionally, one new low severity issue and one informational note were identified and immediately fixed by the developers.

AUDIT PROCESS

The audit was conducted from May 25 to June 2, 2026.

We began by reviewing the provided materials, then met with the development team for a project overview. We performed a comprehensive manual review of all contracts within the audit scope, verifying the logic and security properties, including:

- Standard Solidity issues and common smart contract vulnerabilities;
- Integration with Uniswap V3;
- Whether borrowers could provide insufficient collateral while borrowing excessive amounts;
- Correctness of interest calculation and distribution of Uniswap fees;
- Scenarios where liquidity could become permanently locked;
- Potential reentrancy vectors and risks of fund extraction from the pool;
- Compatibility and risk assessment of non-standard ERC-20 tokens.

We analyzed the project with the following automated tools:

- AI scanner AuditAgent;
- AI scanner Savant Chat;
- AI scanner Guardix.

We executed the test suite. However, we were unable to calculate code coverage.

All identified issues — whether detected manually or through automated tools — were compiled into the private report.

The developers acknowledged the report and implemented changes based on its findings.

We then performed a recheck from June 09 to June 12, 2026. We verified the implemented fixes and reviewed the updated code. Some findings were discussed with the development team and either confirmed as not fixed or supplemented with additional comments. We also discovered one new low severity issue and made one note in the updated codebase.

Finally, we re-ran the tests and updated the report.

MANUAL ANALYSIS

All contracts were manually inspected to validate their logic and security. Additionally, automated analysis results were manually verified. Confirmed issues are documented below.

Critical issues

Critical issues pose severe risks to project security, potentially leading to fund loss or other catastrophic failures. Contracts should not be deployed until such issues are resolved.

C01. Reentrancy risks during minting and interaction with malicious tokens (fixed)

The codebase contains several reentrancy patterns. Consider adding the nonReentrant modifier to the following parts of the code:

- The `NonfungibleLendManager.mint` function violates the Checks-Effects-Interactions pattern due to calling `_safeMint` before state updates.

  Since `_safeMint` triggers `onERC721Received` for `msg.sender`, a malicious liquidity provider can reenter the **NonfungibleLendManager** contract through `provide` or `mint` functions. This reentrancy allows the attacker to increase the recorded liquidity in their position while the actual underlying liquidity remains unchanged.

  As a result, the internal accounting becomes inconsistent, which may allow an attacker to withdraw liquidity belonging to other users in the pool.

- The `Kommodo.open` function reads `balanceBefore`, then performs an external call via `safeTransferFrom`, and only afterwards executes validation checks and updates storage state based on the difference between pre- and post-transfer balances.

  If one of the tokens is malicious, the `safeTransferFrom` call can trigger a reentrancy back into the `open` function. In such a case, the **NonfungibleLendManager** contract may incorrectly update state with a larger collateral amount than was actually transferred, allowing the attacker to borrow more liquidity than they are entitled to.

  Although this scenario requires a malicious token pair, such pools are still possible in Uniswap, as it does not enforce restrictions on token implementation behavior.

- `The NonfungibleLendManager.provide` function reads `pre_liquidity` and then calls `Kommodo.provide`, which internally adds liquidity to Uniswap via `pool.mint`. During `pool.mint`, Uniswap triggers the `uniswapV3MintCallback` in the **Connector** contract.

  Inside this callback, `safeTransferFrom` is used to collect tokens from the lender to Uniswap. If one of the tokens is malicious, the `safeTransferFrom` call inside the callback can trigger a reentrancy back into the `NonfungibleLendManager.provide` function. In such a case, the attacker can manipulate execution flow causing the final `post_liquidity - pre_liquidity` calculation to be incorrect.

  This may result in the liquidity recorded in the NFT position not matching the actual liquidity added to Uniswap. This can allow the attacker to withdraw liquidity belonging to other users.

> `Comment from the developers:`
>
> Only scenario 1 for **NonfungibleLendManager** is a valid critical reentrancy. The other two rely on non supported ERC20 implementations where the malicious tokens manipulate amounts. Because of the importance we implemented reentrancy protection for all of **Kommodo** and **NonfungibleLendManager**.

> `Comment from Pessimistic:`
>
> The issues, including issues 2 and 3 which were classified as non-critical, have been fixed and are not present in the latest version of the code.

Medium severity issues

Medium severity issues may impact the project's current functionality. This category includes bugs, potential revenue loss, operational inefficiencies, and risks related to improper system management. We strongly recommend addressing these issues.

M01. New liquidity providers can capture historical borrower interest (commented)

The **Kommodo** contract distributes historically accrued borrower interest in both `close` and `adjust` (via `setInterest`) functions using the current tick liquidity as the denominator (see `close` at lines 194–195 and `setInterest` at lines 227, 229).

The new lenders can add liquidity immediately before these functions are executed. Since interest is distributed at settlement time using the current `_assets.liquidity`, these late-added liquidity providers are included in the reward base even though they did not participate during the interest accrual period.

As a result, newly added lenders can capture a share of historical borrower interest, reducing the yield of the original liquidity providers.

> `Comment from the developers:`
>
> Acknowledged. This is Just In Time (JIT) Liquidity. Also known in the context of Uniswap swaps where passive liquidity providers lose swap fees to JIT liquidity providers. JIT in the context of **Kommodo** has a different effect because it is liquidity for loans not swaps. The result of JIT on fees is inherent to the way loan interest can be collected onchain. Changing this to other lending market interest designs would result in borrow positions accumulating interest removing the solvency guarantee. The solvency guarantee is the core requirement of the protocol. **Kommodo** implements two strong mitigations:
>
> 1. Liquidity deposits are locked for 1 block so no flash loan JIT liquidity can be provided.
> 2. Added the updateInterest function. UpdateInterest allows anyone including liquidity providers to keep the fee update small, minimizing or in some cases even removing the benefit of JIT. This is no guarantee but a strong defense for the liquidity providers it could hurt.

> `Comment from Pessimistic:`
>
> In the updated version of code, the `Kommodo.updateInterest` function has been introduced, allowing interest to be periodically updated before settlement in close and adjust. However, this does not fully mitigate the issue, as it does not prevent liquidity front-running. New liquidity providers can still be added immediately before settlement and participate in the fee distribution, capturing a portion of historically accrued interest.
>
> Additionally, the existence of the `Kommodo.updateInterest` function does not guarantee that it will ever be called prior to settlement.

M02. New liquidity providers can claim historical Uniswap fees (fixed)

The `Kommodo.updateFeeGrowth` function skips fee synchronization when `_assets.liquidity == _assets.locked`.

When a new lender enters via `Kommodo.provide`, the function calls `Kommodo.updateFeeGrowth` internally. Since `_assets.liquidity - _assets.locked == 0` at that moment, fee synchronization is skipped and the historical Uniswap V3 fees are not accrued before the new lender is added to the liquidity base.

As a result, the newly added lender becomes eligible for Uniswap V3 fees that were earned before their liquidity was deposited, reducing rewards for the original liquidity provider.

> The issue has been fixed and is not present in the latest version of the code.

M03. Provider's position is not synchronized with latest Uniswap fees (fixed)

In the `NonfungibleLendManager.withdraw` function, the position is updated using `pool.assets`, which returns `feeGrowth0X128` and `feeGrowth1X128` values corresponding to the last time the pool was synchronised with Uniswap V3.

However, these values may be outdated at the moment `withdraw` is called, since additional trading fees could have accrued in Uniswap V3 between the last interaction and the current withdrawal. As a result, the position does not fully reflect all fees earned up to the withdrawal time.

After the position update, `Kommodo.withdraw` is called and it collects fees from Uniswap V3 and updates internal accounting (`_withdraws.amountA` and `_withdraws.amountB`). However, the actual transfer is limited to `min(requestedAmount, _withdraws.amount)`, meaning that any excess accrued fees remain unclaimed in the **Kommodo** contract's balance.

Since the **NonfungibleLendManager** contract is a shared wrapper for multiple liquidity providers, these unallocated fees are not attributed to any specific position. As a result, the leftovers may be claimed by a subsequent `withdraw` caller.

> The issue has been fixed and is not present in the latest version of the code.

M04. Typo in mint refund condition (fixed)

In the `NonfungibleLendManager.mint` function, the refund logic incorrectly uses `balanceA` as the condition for refunding both assetA and assetB at lines 91-92.

As a result, assetB is only refunded when `balanceA > 0`, which can lead to assetB remaining stuck in the **NonfungibleLendManager** contract when `balanceA == 0`.

This stranded assetB balance is not attributed to any position and remain in the **NonfungibleLendManager** contract balance, allowing a later provider sweep them.

> The issue has been fixed and is not present in the latest version of the code.

M05. **Insufficient documentation** (commented)

The codebase of the project is well-commented. However, the comments provide only a brief description of the contract functionality and do not explain the overall protocol architecture, assumptions, and limitations:

- For instance, it is not explicitly documented that liquidity providers may remain locked in a specific tick for an extended period of time if there is insufficient available liquidity to withdraw positions.

- The documentation briefly states that non-standard ERC20 tokens are not supported. However, it is unclear how this assumption should be applied in practice, especially since popular assets such as USDT are widely used in Uniswap V3 pools despite having non-standard behavior.

  For example, the protocol uses a custom approval implementation in `NonfungibleLendManager.poolApprove`. Consider using safeApprove or preferably forceApprove from OpenZeppelin **SafeERC20**, which safely handles tokens that require resetting allowances before updating them.

  Additionally, `Kommodo.adjust` performs an unconditional token transfer even when `amountCol == 0`. A borrower may legitimately use `adjust` to partially repay a loan without withdrawing collateral, resulting in a zero-value transfer. While standard ERC20 tokens allow such transfers, some non-standard tokens revert when transferring zero tokens.

The documentation is required to streamline both development and audit processes. It should explicitly explain the purpose and behavior of the contracts, their interactions, supported token assumptions, protocol limitations, and main design choices.

> `Comment from the developers:`
>
> Agreed. Added OpenZeppelin `forceApprove`, checks for non zero transfer amounts and docs for protocol architecture & technical foundations. The documentation is work in progress and will be updated even after mainnet deployment based on user experience/questions. In addition it is fully open source and anyone can contribute.

> `Comment from Pessimistic:`
>
> The protocol's handling of non-standard tokens is now clearer from the code changes, addressing the concerns related to USDT and tokens that revert on zero-value transfers. However, documentation remains important, and the liquidity fragmentation limitation is still not documented.

Low severity issues

Low severity issues do not immediately affect project operations but may introduce risks in future iterations. We recommend resolving these issues or providing justification for the chosen implementation.

L01. Division by zero (fixed)

`Kommodo.getLoanEnd` can revert due to division by zero when `interest` or `colAmount` is zero.

> The issue has been fixed and is not present in the latest version of the code.

L02. Double check (fixed)

Both the **Connector** and **KommodoFactory** contracts perform `_factory != address(0)` checks, which is redundant in properly deployed contracts.

> The issue has been fixed and is not present in the latest version of the code.

L03. Double rounding (fixed)

In the examples below, the calculations are slightly inaccurate due to double rounding in the formulas:

- The `Kommodo.getInterest` function applies `mulDivRoundingUp` twice (lines 312–313), effectively double-rounding the result instead of performing a single precise calculation. This can slightly overstate interest for small values, leading to faster than expected accrual and potentially earlier loan expiration or liquidation.
- In the `Kommodo.getLoanEnd` function, the result is effectively underestimated twice: first, the `yearly` value is rounded up via `mulDivRoundingUp`, which increases the intermediate denominator. And then the final computation applies `mulDiv` (rounding down). As a result, both steps bias the calculation downward, reducing the computed loan duration.

> The issues have been fixed and are not present in the latest version of the code.

L04. Immutable variable (fixed)

The contract has variables which can be declared as `immutable` reducing storage reads and improving gas efficiency in the project:

- The `multiplier` and `factory` in the **KommodoFactory**;
- The `tokenA`, `tokenB`, `fee`, `tickSpacing`, and `interest` in the **Kommodo**.

> The issue has been fixed for the **KommodoFactory** contract and is not applicible for the **Kommodo** contract due to proxy pattern.

L05. Inconsistent and misleading naming reduces code readability (partially fixed)

The codebase contains inconsistent and sometimes misleading naming:

- **(fixed)** Error messages are not aligned with function context, e.g. "open: insufficient collateral for borrow" inside the `Kommodo.adjust` function and "adjust: max amount repay" inside the `Kommodo.close` function, which creates confusion about which lifecycle stage is actually being executed.
- Core function names such as `Kommodo.open`, `Kommodo.adjust`, and `Kommodo.take` are also non-standard for lending logic, which makes the protocol flow less intuitive.
- **(fixed)** There is also ambiguity in state naming, for example `locked` is reused in different contexts with different meanings in the **Kommodo** contract. In addition, the term `interest` is overloaded and used both as a general concept and as `loan.interest` in the **Kommodo** contract.

Overall, this inconsistent naming significantly reduces code readability and increases the risk of misinterpretation during integration and audit.

> `Comment from the developers:`
>
> Adjusted the wrong error messages.
>
> Did not change function names. These are different by design because the borrow primitive is significantly different from current lending implementations. The borrow positions are CLPs instead of single tokens. Using the same naming conventions could lead to confusion.
>
> Adjusted interest to rate for the pool variable. Adjusted `locked` to `paused` for the liquidity providers 1 block lock.

L06. Inconsistent token ordering in parameter passing (commented)

In `NonfungibleLendManager.mint`, token slippage limits (`amountMaxA`, `amountMaxB`) are explicitly sorted to match the canonical token ordering (by token address), ensuring correct alignment with tokenA/tokenB in the pool. In `NonfungibleLendManager.provide`, this sorting logic is missing and parameters are passed as-is.

Consider applying the same ordering logic for consistency.

> `Comment from the developers:`
>
> Not an issue. By design in the `mint` call the user also inputs `token0` and `token1`. Since the underlying pool corrects this ordering we also have to correct this for the input amounts. For the `provide` call the user only inputs amounts. `token0` and `token1` are directly derived from the pool. So `provide` has no input assetA and assetB to compare for ordering.

L07. Missing event (fixed)

`Kommodo.setInterest` does not emit an event, reducing on-chain observability of interest updates.

> The issue has been fixed and is not present in the latest version of the code.

L08. Missing functionality (commented)

The **NonfungibleLendManager** contract lacks `open`, `adjust`, and `close` functionality for borrowers.

These functions are used for the loan lifecycle. Their absence reduces protocol clarity and makes external integration and loans tracking less straightforward.

> `Comment from the developers:`
>
> Not an issue. As the name implies (LendManager) this is the contract to manage lender positions (not borrower positions). Lender positions can be locked if there is insufficient liquidity (all borrowed). **NonfungibleLendManager** allows these locked yield earning positions to be traded, mitigating the lock issue.

L09. Moving logic to constructor (fixed)

Consider moving `Connector.initialize` to the constructor to remove the need for an initialization check. This improves gas efficiency and code readability.

> `Comment from Pessimistic:`
>
> The `Connector.initialize` function was renamed as `__Connector_init`. The issue regarding moving initialization logic to the constructor is still relevant. Additionally, the `factory` variable may be considered for declaration as `immutable` to improve gas efficiency.

> The issue has been fixed and is not present in the latest version of the code.

L10. Multiple reads from storage (partially fixed)

Consider declaring local variable for the following storage reading to reduce gas consumption in the project:

- **(fixed)** `getFee(params.colAmount).toUint128()` in the `Kommodo.open` function as it is read multiple times at lines 121, 125, 127, 128, 131, 133, 134;
- `getKey(params.owner, params.tickBor, params.token0)` in the `Kommodo.close` function at lines 181, 192.

> `Comment from the developers:`
>
> Adjusted for `getFee`. Was implemented this way because of stack to deep errors during development, no error for `getFee` in current implementation so adjusted. For `getKey` still required because of stack too deep.

L11. Overcollateralization increases borrower losses (commented)

Interest in the `Kommodo.getInterest` function is accrued based on the collateral amount rather than the borrowed value.

Therefore, borrowers who provide more collateral than required pay higher interest and may lose more funds during liquidation, even though additional collateral does not change the liquidation condition.

> `Comment from the developers:`
>
> Acknowledged. Determining borrow value requires additional logic (equal to the solvency check). The core logic is minimized so it allows collateral > required. The design itself incentivizes only supplying collateral up to the borrow value, since additional collateral does not change the solvency condition. The fee can be seen an extra incentive not to supply more than required collateral.

L12. Redundant check (fixed)

The `params.liquidity > 0` check in `Kommodo.provide` is redundant, as Uniswap V3 already enforces liquidity constraints. Consider removing it or checking before external calls for efficiency.

> The issue has been fixed and is not present in the latest version of the code.

L13. Griefing via rounding in repeated interest updates (fixed)

`Kommodo.updateInterest` is susceptible to a griefing vector due to the rounding-up behavior in interest accrual inside the `getInterest` function.

For assets with small `decimals` value (for instance, BTC 8 decimals), repeatedly calling `updateInterest` in short intervals can cause repeated upward rounding of accrued interest. This may allow an attacker to effectively accelerate interest depletion and potentially reduce the effective loan duration over time.

> The issue has been fixed and is not present in the latest version of the code.

Notes

N01. Unused import (fixed)

The **KommodoFactory** contract includes an import of **Kommodo.sol**, which is no longer required due to the transition to a proxy-based deployment pattern.

> The issue has been fixed and is not present in the latest version of the code.

This analysis was performed by Pessimistic:

Evgeny Marchenko, Senior Security Engineer
Daria Korepanova, Senior Security Engineer
Evgeny Bokarev, Security Engineer
Irina Vikhareva, Project Manager
Alexander Seleznev, CEO

June 12, 2026